

SAPIENZA
UNIVERSITÀ DI ROMA

Ottimizzazione su reti

Volume 2 — Alberi e cammini

Alberi ricoprenti di costo minimo (Kruskal, Prim, proprietà del taglio e del ciclo) e cammini minimi (grafi aciclici, Bellman–Ford, Dijkstra), con i potenziali come certificato di ottimalità.

Fabio Furini

fabiofurini.github.io

Dipartimento di Ingegneria informatica, automatica e gestionale — Sapienza Università di Roma

Versione online e materiale scaricabile:
fabiofurini.github.io/ottimizzazione-su-reti/
github.com/fabiofurini/ottimizzazione-su-reti

Anno accademico 2026–2027

Indice

2 Alberi ricoprenti di costo minimo **2**

2.1 Il problema dell'albero ricoprente di costo minimo . . . 3
2.2 Tagli e cicli . . . 5
2.3 La proprietà del taglio e la proprietà del ciclo . . . 6
2.4 L'algoritmo greedy generico . . . 9
2.5 Algoritmo di Kruskal . . . 10
2.6 Algoritmo di Prim . . . 13
2.7 Unicità dell'albero ricoprente di costo minimo . . . 15
2.8 Errori tipici . . . 17
2.9 In sintesi . . . 17

3 Cammini minimi **19**

3.1 Problemi di cammino minimo . . . 20
3.2 Condizioni di ottimalità e potenziali . . . 23
3.3 La formulazione di programmazione lineare . . . 25
3.4 Cammini minimi nei grafi aciclici . . . 28
3.5 L'algoritmo di Bellman–Ford . . . 30
3.6 L'algoritmo di Dijkstra . . . 35
3.7 Errori tipici . . . 37
3.8 In sintesi . . . 38

Capitolo 2

Alberi ricoprenti di costo minimo

Prerequisiti

- Il capitolo «Grafi»: cammini e cicli, componenti connesse, le caratterizzazioni di un albero e i cicli fondamentali.
- La notazione $O(\cdot)$ per il tempo di esecuzione di un algoritmo, e l'ordinamento di m numeri in tempo $O(m \log m)$.

Obiettivi

Alla fine di questo capitolo sarai in grado di:

- enunciare il problema dell'albero ricoprente di costo minimo e riconoscerlo nelle applicazioni;
- dimostrare la proprietà del taglio e la proprietà del ciclo, e usarle per certificare che un albero ricoprente è ottimo;
- eseguire gli algoritmi di Kruskal e di Prim, dimostrarne la correttezza e analizzarne il tempo di esecuzione;
- stabilire se l'albero ricoprente di costo minimo è unico, e trovare gli spigoli che appartengono a ogni albero ottimo.

2.1 Il problema dell'albero ricoprente di costo minimo

Definizione 2.1: problema dell'albero ricoprente di costo minimo

Sia $G = (V, E)$ un grafo non orientato connesso con $n = |V|$ vertici e $m = |E|$ spigoli, e sia associato a ogni spigolo $e \in E$ un **peso** $w_e \in \mathbb{R}$. Il **peso** di un insieme di spigoli $F \subseteq E$ è

$$w(F) = \sum_{e \in F} w_e.$$

Il **problema dell'albero ricoprente di costo minimo** (MST) chiede un albero ricoprente $\tilde{T} = (V, \tilde{F})$ di G di peso minimo:

$$w(\tilde{F}) \leq w(F) \qquad \text{per ogni albero ricoprente } (V, F) \text{ di } G.$$

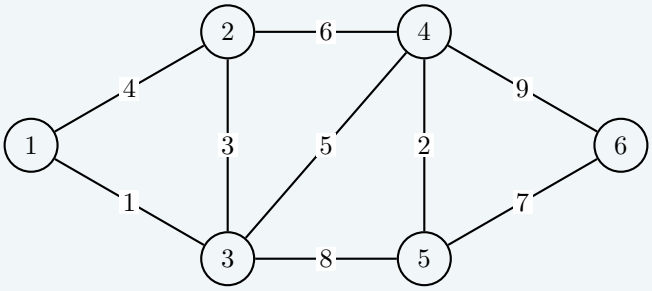
Un albero ricoprente di peso minimo è un **albero ricoprente di costo minimo**. Scriviamo w_{uv} per il peso dello spigolo $\{u, v\}$, e identifichiamo un albero ricoprente con il suo insieme di spigoli.

- Un grafo connesso ha almeno un albero ricoprente, e ne ha un numero finito: un albero ricoprente di costo minimo esiste sempre. Ogni albero ricoprente ha $n - 1$ spigoli.
- I pesi possono essere negativi. Aggiungere la stessa costante a ogni peso aggiunge $(n-1)$ volte quella costante al peso di ogni albero ricoprente, quindi gli alberi ricoprenti di costo minimo non cambiano.
- Un **albero ricoprente di costo massimo** è un albero ricoprente di costo minimo per i pesi $-w_e$.

Esempio 2.1: il grafo di riferimento pesato

Prendiamo il grafo non orientato soggiacente al grafo di riferimento del capitolo «Grafi», con $n = 6$ vertici, $m = 9$ spigoli e i pesi

spigolo	$\{1,2\}$	$\{1,3\}$	$\{2,3\}$	$\{2,4\}$	$\{3,4\}$	$\{3,5\}$	$\{4,5\}$	$\{4,6\}$	$\{5,6\}$
w_e	4	1	3	6	5	8	2	9	7

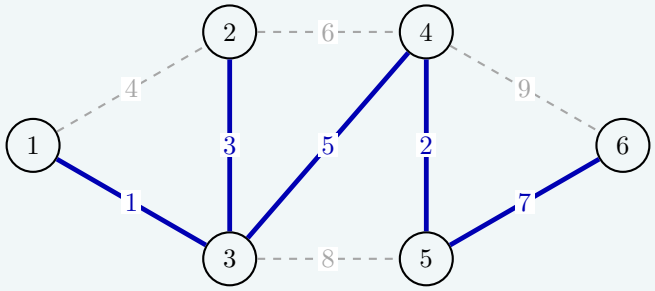


Chiamiamo questo grafo **grafo di riferimento pesato**: tutti gli esempi del capitolo lo usano. I pesi sono gli interi distinti $1, 2, \ldots, 9$.

Esempio 2.2: l'albero ricoprente di costo minimo del grafo di riferimento

Il grafo di riferimento pesato ha 55 alberi ricoprenti. Enumerandoli tutti, il peso minimo è 18, raggiunto soltanto dall'albero (in blu nella figura)

$$\tilde{F} = \big\{\{1,3\},\{2,3\},\{3,4\},\{4,5\},\{5,6\}\big\}, \qquad w(\tilde{F}) = 1+3+5+2+7 = 18.$$



L'albero ricoprente $\{1,2\},\{1,3\},\{2,4\},\{3,5\},\{4,6\}$ trovato dalla visita in ampiezza nel capitolo «Grafi» pesa $4+1+6+8+9 = 28$, e un albero ricoprente di costo massimo pesa 34.

- Enumerare gli alberi ricoprenti è praticabile solo su grafi piccolissimi: il grafo di riferimento, con 6 vertici e 9 spigoli, ne ha già 55. Gli algoritmi di questo capitolo trovano un albero ricoprente di costo minimo in tempo polinomiale.

2.1.1 Applicazioni

- **Progettazione di reti.** I vertici sono siti (uffici, case, stabilimenti) e w_{uv} è il costo di un collegamento diretto (un cavo, un tubo, una strada); la rete più economica che collega tutti i siti è un albero ricoprente di costo minimo.
- **Clustering.** Togliendo i $k-1$ spigoli più pesanti di un albero ricoprente di costo minimo, i vertici si dividono in k gruppi di punti vicini (clustering a collegamento singolo).
- **Sottoproblema.** Gli alberi ricoprenti di costo minimo danno lower bound per il problema del commesso viaggiatore e un punto di partenza per algoritmi euristici di progettazione di reti.

Un albero ricoprente di costo minimo collega tutti i vertici di un grafo pesato connesso con $n-1$ spigoli di peso totale minimo; sul grafo di riferimento pesato è unico e pesa 18.

2.2 Tagli e cicli

Definizione 2.2: taglio

Sia $G = (V, E)$ un grafo non orientato e $S \subseteq V$ con $\emptyset \neq S \neq V$. Il **taglio** definito da S è l'insieme degli spigoli con esattamente un estremo in S:

$$\delta(S) = \{\, \{u, v\} \in E : u \in S,\ v \notin S \,\}.$$

Uno spigolo di $\delta(S)$ **attraversa** il taglio. Per un singolo vertice, $\delta(\{v\}) = \delta(v)$ è l'insieme degli spigoli incidenti in v.

- Un grafo è connesso se e solo se ogni taglio $\delta(S)$ è non vuoto: un taglio vuoto separa S da $V \setminus S$, e un cammino da $u \in S$ a $v \notin S$ deve usare uno spigolo di $\delta(S)$.
- Per esempio, nel grafo di riferimento pesato, $\delta(\{1, 2, 3\}) = \{\{2, 4\}, \{3, 4\}, \{3, 5\}\}$ e $\delta(\{6\}) = \{\{4, 6\}, \{5, 6\}\}$.

Lemma 2.3: un ciclo attraversa un taglio un numero pari di volte

Sia C un ciclo di G e $\delta(S)$ un taglio. Allora $|C \cap \delta(S)|$ è pari. In particolare, se un ciclo contiene uno spigolo di $\delta(S)$, contiene almeno un altro spigolo di $\delta(S)$.

Dimostrazione. Siano $v_0, v_1, \ldots, v_k = v_0$ i vertici visitati dal ciclo. Lo spigolo $\{v_{t-1}, v_t\}$ attraversa il taglio se e solo se v_{t-1} e v_t stanno da parti diverse. Percorrendo il ciclo, la parte cambia a ogni spigolo che attraversa il taglio, e solo lì; il percorso termina in v_0, dalla parte da cui era partito, quindi il numero di cambi è pari. □

Definizione 2.4: taglio fondamentale

Sia T un albero ricoprente di un grafo connesso $G = (V, E)$ ed $e = \{u, v\} \in T$. Il grafo $T \setminus \{e\}$ ha due componenti connesse; sia S_e quella che contiene u. Il **taglio fondamentale** di e è $\delta(S_e)$.

- $T \setminus \{e\}$ ha $n - 2$ spigoli e nessun ciclo, quindi ha $n - (n - 2) = 2$ componenti (Corollario «spigoli e componenti»); v non sta in S_e, altrimenti il cammino da u a v in T non sarebbe unico.
- L'unico spigolo di T nel taglio fondamentale di e è e stesso: ogni altro spigolo di T sta dentro una delle due componenti.

Lemma 2.5: scambio

Sia T un albero ricoprente di un grafo connesso $G = (V, E)$.

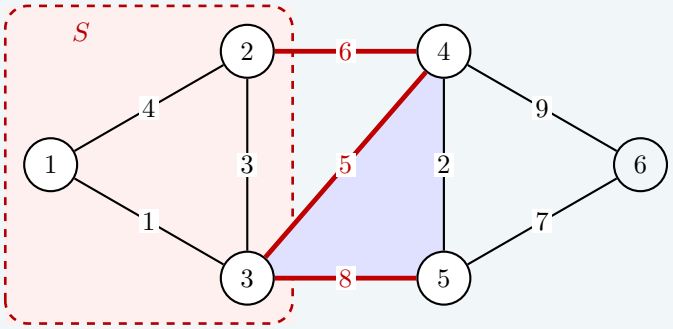
1. Se $e \notin T$ e f è uno spigolo del ciclo fondamentale $C(T, e)$, allora $T \cup \{e\} \setminus \{f\}$ è un albero ricoprente.
2. Se $f \in T$ ed $e \notin T$ attraversa il taglio fondamentale di f, allora $f \in C(T, e)$, e $T \cup \{e\} \setminus \{f\}$ è un albero ricoprente.

Dimostrazione. 1. Il resto di $C(T,e)$ collega gli estremi di f, quindi togliendo f dal grafo connesso $T \cup \{e\}$ si ottiene ancora un grafo connesso. Il risultato è connesso e ha $n-1$ spigoli: è un albero ricoprente per le caratterizzazioni di un albero, (b).

2. Gli estremi di e stanno da parti diverse del taglio fondamentale $\delta(S_f)$. Il cammino fra loro in T fa parte di $C(T,e)$, e deve attraversare $\delta(S_f)$ con uno spigolo di T; l'unico è f. Il punto 1 conclude. □

Esempio 2.3: tagli e cicli del grafo di riferimento

La figura mostra il taglio $\delta(\{1,2,3\})$ (spigoli rossi) e il ciclo $3,4,5$ (ombreggiato in blu): il ciclo usa gli spigoli $\{3,4\}$ e $\{3,5\}$ del taglio, un numero pari.



Per l'albero ricoprente di costo minimo $\tilde{F}$ dell'Esempio 2.2, il taglio fondamentale di $\{3,4\}$ è $\delta(\{1,2,3\})$, e il ciclo fondamentale di $\{3,5\}$ è $C(\tilde{F},\{3,5\}) = 3,4,5,3$.

Un ciclo attraversa ogni taglio un numero pari di volte. Aggiungendo uno spigolo a un albero ricoprente e togliendo un qualsiasi spigolo del ciclo che esso chiude, si ottiene un altro albero ricoprente.

2.3 La proprietà del taglio e la proprietà del ciclo

Teorema 2.6: proprietà del taglio

Sia $G = (V,E)$ un grafo connesso con pesi w_e, e sia $F \subseteq E$ contenuto in qualche albero ricoprente di costo minimo. Sia $\delta(S)$ un taglio con $F \cap \delta(S) = \emptyset$, e sia e uno spigolo di peso minimo in $\delta(S)$.

(a) $F \cup \{e\}$ è contenuto in qualche albero ricoprente di costo minimo.

(b) Se $w_e < w_f$ per ogni $f \in \delta(S) \setminus \{e\}$, allora e appartiene a ogni albero ricoprente di costo minimo.

Dimostrazione. Sia $T^\star$ un albero ricoprente di costo minimo con $F \subseteq T^\star$; per (b), sia $T^\star$ un qualsiasi albero ricoprente di costo minimo e $F = \emptyset$. Se $e \in T^\star$ non c'è niente da dimostrare; sia $e \notin T^\star$.

1. Il ciclo fondamentale $C(T^\star, e)$ contiene lo spigolo e di $\delta(S)$, quindi contiene un altro spigolo $f \in \delta(S)$, $f \neq e$ (Lemma 2.3). Tutti i suoi spigoli diversi da e stanno in $T^\star$, quindi $f \in T^\star$; e $f \notin F$, perché $F \cap \delta(S) = \emptyset$.
2. Per il Lemma 2.5, $T' = T^\star \cup \{e\} \setminus \{f\}$ è un albero ricoprente, con $w(T') = w(T^\star) + w_e - w_f$.
3. (a) e ha peso minimo in $\delta(S)$, quindi $w_e \leq w_f$ e $w(T') \leq w(T^\star)$: T' è un albero ricoprente di costo minimo che contiene $F \cup \{e\}$.
4. (b) Qui $w_e < w_f$ e $w(T') < w(T^\star)$, contro l'ottimalità di $T^\star$: quindi $e \in T^\star$. □

Teorema 2.7: proprietà del ciclo

Sia $G = (V, E)$ un grafo connesso con pesi w_e, sia C un ciclo di G ed e uno spigolo di peso massimo in C.

(a) Qualche albero ricoprente di costo minimo non contiene e.

(b) Se $w_e > w_f$ per ogni $f \in C \setminus \{e\}$, allora nessun albero ricoprente di costo minimo contiene e.

Dimostrazione. Sia $T^\star$ un albero ricoprente di costo minimo con $e \in T^\star$ (se non ce n'è nessuno, entrambe le affermazioni valgono), e sia $\delta(S_e)$ il taglio fondamentale di e in $T^\star$.

1. Il ciclo C contiene lo spigolo e di $\delta(S_e)$, quindi contiene un altro spigolo $f \in \delta(S_e)$ (Lemma 2.3); $f \notin T^\star$, perché e è l'unico spigolo di $T^\star$ nel suo taglio fondamentale.
2. Per il Lemma 2.5(2), $T' = T^\star \cup \{f\} \setminus \{e\}$ è un albero ricoprente, con $w(T') = w(T^\star) + w_f - w_e$.
3. (a) $w_f \leq w_e$, quindi T' è un albero ricoprente di costo minimo senza e. (b) $w_f < w_e$ darebbe $w(T') < w(T^\star)$, una contraddizione: nessun albero ricoprente di costo minimo contiene e. □

Esempio 2.4: le due proprietà sul grafo di riferimento

- *Proprietà del taglio.* Lo spigolo $\{3, 4\}$, di peso 5, è l'unico spigolo più leggero di $\delta(\{1, 2, 3\}) = \{\{2, 4\}, \{3, 4\}, \{3, 5\}\}$, con pesi $6, 5, 8$: appartiene a ogni albero ricoprente di costo minimo. Allo stesso modo, $\{5, 6\}$ è l'unico spigolo più leggero di $\delta(\{6\})$.
- *Proprietà del ciclo.* Lo spigolo $\{1, 2\}$ è l'unico spigolo più pesante del ciclo $1, 2, 3$ (pesi $4, 3, 1$), e $\{3, 5\}$ è l'unico spigolo più pesante del ciclo $3, 4, 5$ (pesi $5, 2, 8$): nessun

albero ricoprente di costo minimo li contiene. I cicli $2,3,4$ e $4,5,6$ escludono $\{2,4\}$ e $\{4,6\}$.

- I quattro spigoli esclusi lasciano esattamente i 5 spigoli di $\tilde{F}$: le due proprietà da sole dimostrano che $\tilde{F}$ è l'unico albero ricoprente di costo minimo.

2.3.1 Condizioni di ottimalità

Teorema 2.8: condizioni di ottimalità

Sia T un albero ricoprente di un grafo connesso $G = (V, E)$ con pesi w_e. Le seguenti affermazioni sono equivalenti:

(i) T è un albero ricoprente di costo minimo;

(ii) **condizione del ciclo**: per ogni $e \notin T$, $w_e \geq w_f$ per ogni spigolo f di $C(T, e)$;

(iii) **condizione del taglio**: per ogni $e \in T$, $w_e \leq w_f$ per ogni spigolo f del taglio fondamentale di e.

Dimostrazione. Dimostriamo (i)⇒(iii)⇒(ii)⇒(i).

- **(i)⇒(iii).** Se f attraversa il taglio fondamentale di $e \in T$ con $w_f < w_e$, allora $T \cup \{f\} \setminus \{e\}$ è un albero ricoprente (Lemma 2.5(2)) più leggero di T.
- **(iii)⇒(ii).** Siano $e \notin T$ e $f \in C(T, e)$, $f \neq e$. Il cammino di T fra gli estremi di e usa f, ed è unico, quindi togliere f li separa: e attraversa il taglio fondamentale di f, e $w_f \leq w_e$ per (iii).

(ii)⇒(i). Fra gli alberi ricoprenti di costo minimo, sia $T^\star$ uno con il massimo numero di spigoli in comune con T, e supponiamo $T^\star \neq T$.

1. Prendiamo $e \in T^\star \setminus T$, e sia $\delta(S_e)$ il suo taglio fondamentale in $T^\star$. Il ciclo $C(T, e)$ contiene e, quindi contiene un altro spigolo $f \in \delta(S_e)$ (Lemma 2.3), con $f \in T$ e $f \notin T^\star$.
2. Per il Lemma 2.5(2), $T' = T^\star \cup \{f\} \setminus \{e\}$ è un albero ricoprente, e $w_f \leq w_e$ per (ii) applicata a $e \notin T$. Quindi $w(T') \leq w(T^\star)$.
3. T' è un albero ricoprente di costo minimo con uno spigolo in più in comune con T: una contraddizione. Quindi $T = T^\star$. □

Esempio 2.5: certificare l'albero ottimo

Per l'albero $\tilde{F}$ dell'Esempio 2.2, i quattro spigoli fuori dall'albero e i loro cicli fondamentali sono:

spigolo $e \notin \tilde{F}$	$C(\tilde{F}, e)$	pesi degli spigoli dell'albero nel ciclo
$\{1,2\}$, $w=4$	$1,3,2,1$	$1,\ 3$
$\{2,4\}$, $w=6$	$2,3,4,2$	$3,\ 5$
$\{3,5\}$, $w=8$	$3,4,5,3$	$5,\ 2$
$\{4,6\}$, $w=9$	$4,5,6,4$	$2,\ 7$

Ogni spigolo fuori dall'albero è pesante almeno quanto gli spigoli dell'albero del suo ciclo: per il Teorema 2.8 l'albero è ottimo, senza enumerare i 55 alberi ricoprenti.

Esempio 2.6: migliorare un albero che non è ottimo

L'albero della visita in ampiezza $T = \{\{1,2\},\{1,3\},\{2,4\},\{3,5\},\{4,6\}\}$, di peso 28, viola la condizione del ciclo. Per $e = \{4,5\}$, di peso 2, il ciclo fondamentale $4,2,1,3,5,4$ contiene $\{2,4\}$, $\{1,2\}$ e $\{3,5\}$, di pesi $6,4,8$.

- Togliendo il più pesante dei tre, $\{3,5\}$, e aggiungendo $\{4,5\}$ si ottiene un albero ricoprente di peso $28+2-8=22$.
- La dimostrazione del Teorema 2.8 diventa un algoritmo: ripetere lo scambio finché uno spigolo viola la condizione. Gli algoritmi delle prossime sezioni sono più veloci.

Lo spigolo più leggero di un taglio appartiene a qualche albero ricoprente di costo minimo, e lo spigolo più pesante di un ciclo si può lasciare fuori. Un albero ricoprente è ottimo se e solo se ogni spigolo fuori dall'albero è il più pesante del suo ciclo fondamentale.

2.4 L'algoritmo greedy generico

- La proprietà del taglio suggerisce di costruire un albero ricoprente di costo minimo uno spigolo alla volta: l'insieme corrente F è sempre contenuto in qualche albero ricoprente di costo minimo, e lo spigolo successivo è il più leggero di un taglio non attraversato da F.

Algoritmo

Algorithm: Algoritmo greedy generico per l'MST

Input: un grafo non orientato connesso $G=(V,E)$ con pesi w_e
Output: un albero ricoprente di costo minimo F

1 **Passo 0 (inizializzazione).** Porre $F := \emptyset$.
2 **Passo 1 (test di arresto).** Se $|F| = n-1$, `STOP`.
3 **Passo 2 (aumento).** Scegliere un taglio $\delta(S)$ con $F \cap \delta(S) = \emptyset$, e aggiungere a F uno spigolo di peso minimo in $\delta(S)$. Andare al Passo 1.

Teorema 2.9: correttezza dell'algoritmo greedy generico

A ogni iterazione dell'algoritmo greedy generico esiste un taglio non vuoto come nel Passo 2, e F è contenuto in qualche albero ricoprente di costo minimo. Alla fine, F è un albero ricoprente di costo minimo.

Dimostrazione. Per induzione sulle iterazioni; $F = \emptyset$ è contenuto in ogni albero ricoprente di costo minimo.

1. Se $|F| < n - 1$, il grafo (V, F) è una foresta (un sottoinsieme di un albero) con $n - |F| \geq 2$ componenti. Sia S una di esse: nessuno spigolo di F attraversa $\delta(S)$, e $\delta(S) \neq \emptyset$ perché G è connesso.
2. Per la proprietà del taglio (Teorema 2.6(a)), il nuovo insieme $F \cup \{e\}$ è ancora contenuto in qualche albero ricoprente di costo minimo.
3. Quando $|F| = n - 1$, F è contenuto in un albero ricoprente di costo minimo $T^\star$ con $n - 1$ spigoli, quindi $F = T^\star$. □

- Lo spigolo aggiunto $e = \{u, v\}$ ha $u \in S$ e $v \notin S$, e nessuno spigolo di F attraversa $\delta(S)$: u e v stanno in componenti diverse di (V, F), quindi e non chiude un ciclo.
- Gli algoritmi di Kruskal e di Prim sono due modi di scegliere il taglio nel Passo 2.

Aggiungendo $n - 1$ volte lo spigolo più leggero di un taglio non attraversato dagli spigoli correnti si costruisce un albero ricoprente di costo minimo.

2.5 Algoritmo di Kruskal

Algoritmo

Algorithm: Algoritmo di Kruskal

Input: un grafo non orientato connesso $G = (V, E)$ con pesi w_e
Output: un albero ricoprente di costo minimo F

1 **Passo 0 (inizializzazione).** Ordinare gli spigoli per peso non decrescente, $w_{e_1} \leq w_{e_2} \leq \cdots \leq w_{e_m}$, e porre $F := \emptyset$.
2 **Passo 1 (scansione).** Per $k = 1, 2, \ldots, m$: se gli estremi di e_k stanno in componenti diverse di (V, F), porre $F := F \cup \{e_k\}$; altrimenti rifiutare e_k.
3 Se $|F| = n - 1$, `STOP`.

- Uno spigolo è **accettato** se unisce due componenti diverse di (V, F), e **rifiutato** se chiuderebbe un ciclo. Le componenti di (V, F) partono come gli n vertici isolati e si fondono a ogni accettazione.
- In questo modulo, gli spigoli di peso uguale si esaminano in ordine lessicografico di $\{u, v\}$, con $u < v$.

Teorema 2.10: correttezza dell'algoritmo di Kruskal

L'algoritmo di Kruskal si ferma con $|F| = n - 1$, e F è un albero ricoprente di costo minimo.

Dimostrazione. Mostriamo che ogni spigolo accettato $e_k = \{u, v\}$ è una scelta del Passo 2 dell'algoritmo greedy generico. Sia S la componente di (V, F) che contiene u, subito prima che e_k sia accettato: $v \notin S$, e nessuno spigolo di F attraversa $\delta(S)$.

1. Sia $e_h \in \delta(S)$ con $h < k$. Quando e_h è stato esaminato non è stato accettato, perché $F \cap \delta(S) = \emptyset$; quindi i suoi estremi stavano allora nella stessa componente, e ci stanno ancora, perché le componenti si fondono soltanto. Questo contraddice $e_h \in \delta(S)$.
2. Quindi ogni spigolo di $\delta(S)$ è un e_h con $h \geq k$, e $w_{e_h} \geq w_{e_k}$: e_k ha peso minimo in $\delta(S)$.
3. Se la scansione terminasse con $|F| < n - 1$, la foresta (V, F) avrebbe due componenti unite da uno spigolo di G, e quello spigolo sarebbe stato accettato quando è stato esaminato, per l'argomento del punto 1.

Per il Teorema 2.9, F è un albero ricoprente di costo minimo. □

2.5.1 Implementazione e tempo di esecuzione

- Le componenti di (V, F) sono mantenute in una struttura **union–find**: ogni vertice v memorizza l'etichetta $comp(v)$ della sua componente, e ogni componente memorizza la lista dei suoi vertici e la sua dimensione.
- Verificare se u e v stanno in componenti diverse è il confronto $comp(u) \neq comp(v)$, in tempo $O(1)$. Fondere due componenti rietichetta i vertici della *più piccola* (**unione per dimensione**).

Teorema 2.11: tempo di esecuzione dell'algoritmo di Kruskal

Con la struttura union–find per etichette e l'unione per dimensione, l'algoritmo di Kruskal richiede tempo $O(m \log n)$.

Dimostrazione.
1. Ordinare gli spigoli richiede tempo $O(m \log m)$; poiché $m \leq n^2$, $\log m \leq 2 \log n$ e questo è $O(m \log n)$.
2. La scansione esegue m test, ciascuno in tempo $O(1)$.
3. Quando un vertice viene rietichettato, la sua componente si fonde con una almeno altrettanto grande, quindi la dimensione della sua componente almeno raddoppia. La dimensione non supera mai n, quindi ogni vertice viene rietichettato al più $\log_2 n$ volte: tutte le fusioni richiedono tempo $O(n \log n)$.

G è connesso, quindi $n - 1 \leq m$, e il totale è $O(m \log n + n \log n) = O(m \log n)$. □

- Il tempo di esecuzione è dominato dall'ordinamento. Con la struttura union–find ad alberi, con unione per rango e compressione dei cammini, la scansione richiede un tempo quasi lineare; il limite resta $O(m \log n)$ a causa dell'ordinamento.
- L'algoritmo usa solo confronti fra pesi: ogni trasformazione crescente dei pesi dà lo stesso albero.

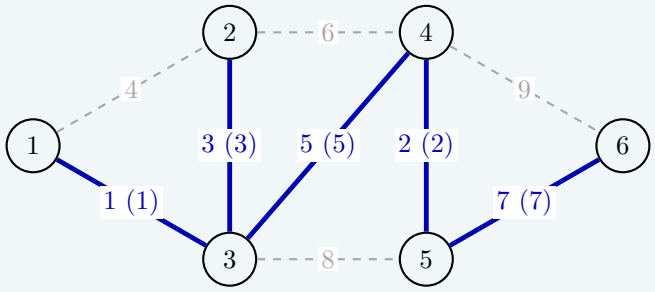
2.5.2 Algoritmo di Kruskal sul grafo di riferimento

Esempio 2.7: algoritmo di Kruskal

Eseguiamo l'algoritmo di Kruskal sul grafo di riferimento pesato. Gli spigoli in ordine di peso sono $\{1,3\}, \{4,5\}, \{2,3\}, \{1,2\}, \{3,4\}, \{2,4\}, \{5,6\}, \{3,5\}, \{4,6\}$. Ogni riga è uno spigolo esaminato, con le componenti di (V, F) dopo il passo:

k	spigolo e_k	w_{e_k}	decisione e componenti di (V, F)
1	$\{1,3\}$	1	accetta: $\{1,3\}, \{2\}, \{4\}, \{5\}, \{6\}$
2	$\{4,5\}$	2	accetta: $\{1,3\}, \{2\}, \{4,5\}, \{6\}$
3	$\{2,3\}$	3	accetta: $\{1,2,3\}, \{4,5\}, \{6\}$
4	$\{1,2\}$	4	rifiuta (1 e 2 stanno entrambi in $\{1,2,3\}$)
5	$\{3,4\}$	5	accetta: $\{1,2,3,4,5\}, \{6\}$
6	$\{2,4\}$	6	rifiuta (2 e 4 stanno entrambi in $\{1,2,3,4,5\}$)
7	$\{5,6\}$	7	accetta: $\{1,2,3,4,5,6\}$; $\lvert F\rvert = 5$, `STOP`

L'algoritmo si ferma dopo 7 dei 9 spigoli, con $F = \{\{1,3\}, \{4,5\}, \{2,3\}, \{3,4\}, \{5,6\}\}$ di peso 18: l'albero $\tilde{F}$ dell'Esempio 2.2. La figura mostra, su ogni spigolo dell'albero, il suo peso e fra parentesi il passo k in cui è stato accettato.



- Gli spigoli rifiutati $\{1,2\}$ e $\{2,4\}$ sono gli spigoli più pesanti dei cicli $1,2,3$ e $2,3,4$, come prevede la proprietà del ciclo; gli spigoli $\{3,5\}$ e $\{4,6\}$ non vengono mai esaminati.

L'algoritmo di Kruskal esamina gli spigoli per peso non decrescente e accetta quelli che uniscono due componenti; con union–find richiede tempo $O(m \log n)$, dominato dall'ordinamento.

2.6 Algoritmo di Prim

- L'algoritmo di Prim fa crescere un solo albero da un vertice di partenza s: l'insieme S dei vertici dell'albero definisce il taglio $\delta(S)$ del Passo 2, e lo spigolo più leggero del taglio si trova attraverso un'**etichetta** $d(v)$ per ogni vertice $v \notin S$.

Algoritmo

Algorithm: Algoritmo di Prim

Input: un grafo non orientato connesso $G = (V, E)$ con pesi w_e, dato con le sue liste di adiacenza; un vertice di partenza $s \in V$

Output: un albero ricoprente di costo minimo F

1 **Passo 0 (inizializzazione).** Porre $S := \{s\}$ e $F := \emptyset$. Per ogni $v \neq s$: se $\{s, v\} \in E$, porre $d(v) := w_{sv}$ e $pred(v) := s$; altrimenti porre $d(v) := +\infty$.

2 **Passo 1 (selezione).** Se $S = V$, `STOP`. Altrimenti selezionare $v \notin S$ con $d(v)$ minimo, e porre $S := S \cup \{v\}$, $F := F \cup \{\{pred(v), v\}\}$.

3 **Passo 2 (aggiornamento).** Per ogni $u \notin S$ adiacente a v con $w_{vu} < d(u)$, porre $d(u) := w_{vu}$ e $pred(u) := v$. Andare al Passo 1.

- In questo modulo il vertice di partenza è $s = 1$, e nel Passo 1, a parità di etichetta, si sceglie il vertice di indice minimo.

Lemma 2.12: significato delle etichette

A ogni esecuzione del Passo 1 dell'algoritmo di Prim, per ogni $v \notin S$,

$$d(v) = \min \{\, w_{uv} : u \in S,\ \{u, v\} \in E \,\}$$

($+\infty$ se v non ha vicini in S), e quando $d(v) < +\infty$ il minimo è raggiunto da $u = pred(v)$.

Dimostrazione. Per induzione. Alla prima esecuzione $S = \{s\}$, e il Passo 0 pone esattamente $d(v) = w_{sv}$, oppure $+\infty$. Supponiamo vera l'affermazione prima dell'inserimento di un vertice v in S. Per $u \notin S \cup \{v\}$, il nuovo insieme $S \cup \{v\}$ aggiunge il candidato w_{vu} quando $\{v, u\} \in E$, e il Passo 2 sostituisce $d(u)$ e $pred(u)$ esattamente quando questo candidato è più piccolo. Le altre etichette e gli altri predecessori non cambiano. □

Teorema 2.13: correttezza dell'algoritmo di Prim

L'algoritmo di Prim si ferma con $|F| = n - 1$, e F è un albero ricoprente di costo minimo.

Dimostrazione. Tutti gli spigoli di F hanno entrambi gli estremi in S, quindi $F \cap \delta(S) = \emptyset$.

1. Per il Lemma 2.12, $\min_{v \notin S} d(v)$ è il peso minimo di uno spigolo di $\delta(S)$, raggiunto da $\{pred(v), v\}$ per il v selezionato; è finito perché G è connesso.
2. Quindi ogni iterazione è una scelta del Passo 2 dell'algoritmo greedy generico,

con il taglio $\delta(S)$.

3. Ogni iterazione aggiunge un vertice a S e uno spigolo a F, quindi l'algoritmo si ferma dopo $n-1$ iterazioni con $|F| = n-1$; il Teorema 2.9 conclude. □

Teorema 2.14: tempo di esecuzione dell'algoritmo di Prim

Con le etichette memorizzate in un array, l'algoritmo di Prim richiede tempo $O(n^2)$.

Dimostrazione. Ci sono $n-1$ selezioni, e ciascuna scorre le etichette di al più n vertici: $O(n^2)$ in totale. L'aggiornamento dopo l'inserimento di v scorre la lista di adiacenza di v in tempo $O(\deg(v))$, e $\sum_{v \in V} \deg(v) = 2m$. Il totale è $O(n^2 + m) = O(n^2)$, poiché $m \leq n^2$. □

- Sui grafi densi, con m dell'ordine di n^2, il limite $O(n^2)$ è lineare nella dimensione del grafo.
- Con le etichette in uno heap binario, ogni selezione e ogni aggiornamento richiedono tempo $O(\log n)$, per $O(m \log n)$ in totale: meglio sui grafi sparsi.

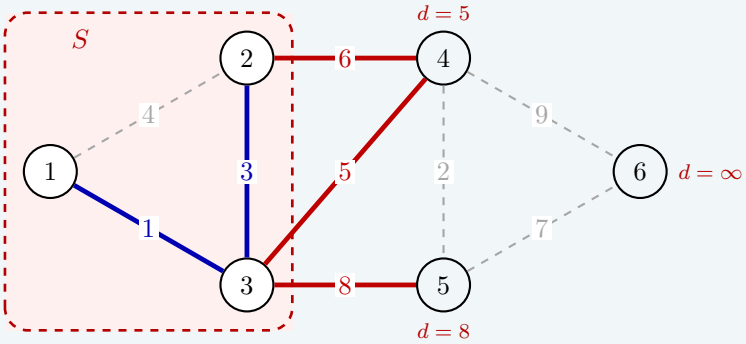
2.6.1 Algoritmo di Prim sul grafo di riferimento

Esempio 2.8: algoritmo di Prim dal vertice 1

Eseguiamo l'algoritmo di Prim sul grafo di riferimento pesato a partire da $s = 1$. Ogni riga mostra il vertice selezionato, lo spigolo aggiunto a F e le etichette $d(v)$ dei vertici fuori da S dopo l'aggiornamento, con $pred(v)$ fra parentesi; un trattino indica i vertici già in S.

iterazione	selezionato	spigolo aggiunto	$d(2)$	$d(3)$	$d(4)$	$d(5)$	$d(6)$
0	1	–	4 (1)	1 (1)	∞	∞	∞
1	3	$\{1,3\}$	3 (3)	–	5 (3)	8 (3)	∞
2	2	$\{2,3\}$	–	–	5 (3)	8 (3)	∞
3	4	$\{3,4\}$	–	–	–	2 (4)	9 (4)
4	5	$\{4,5\}$	–	–	–	–	7 (5)
5	6	$\{5,6\}$	–	–	–	–	–

All'iterazione 1 l'etichetta di 2 scende da 4 a 3, perché $w_{32} = 3 < 4$; all'iterazione 3 l'etichetta di 5 scende da 8 a 2. La figura mostra lo stato dopo l'iterazione 2: il taglio $\delta(\{1,2,3\})$ è attraversato da $\{2,4\}$, $\{3,4\}$, $\{3,5\}$, e il prossimo spigolo è $\{3,4\}$, con $d(4) = 5$.



L'albero finale è $\{1,3\},\{2,3\},\{3,4\},\{4,5\},\{5,6\}$, di peso 18: lo stesso albero dell'algoritmo di Kruskal, con gli spigoli aggiunti in un ordine diverso.

- L'algoritmo di Kruskal aggiunge $\{4,5\}$ per secondo, l'algoritmo di Prim per quarto: Kruskal fa crescere una foresta, Prim un solo albero che contiene s.

L'algoritmo di Prim fa crescere un albero da s e aggiunge lo spigolo più leggero che ne esce; le etichette $d(v)$ danno quello spigolo in tempo $O(n)$, per $O(n^2)$ in totale.

2.7 Unicità dell'albero ricoprente di costo minimo

Teorema 2.15: pesi distinti

Se i pesi w_e, $e \in E$, sono distinti a due a due, il grafo connesso $G = (V, E)$ ha esattamente un albero ricoprente di costo minimo.

Dimostrazione. Supponiamo che $T_1 \neq T_2$ siano alberi ricoprenti di costo minimo. Hanno lo stesso numero di spigoli, quindi la differenza simmetrica $(T_1 \setminus T_2) \cup (T_2 \setminus T_1)$ è non vuota; sia e il suo spigolo più leggero, e supponiamo $e \in T_1 \setminus T_2$.

1. Il ciclo $C(T_2, e)$ non è contenuto in T_1, che è aciclico, quindi ha uno spigolo $f \notin T_1$; $f \neq e$, e $f \in T_2$, quindi $f \in T_2 \setminus T_1$.
2. f sta nella differenza simmetrica e $f \neq e$, quindi $w_f > w_e$, perché i pesi sono distinti.
3. $T_2 \cup \{e\} \setminus \{f\}$ è un albero ricoprente (Lemma 2.5) di peso $w(T_2) + w_e - w_f < w(T_2)$: una contraddizione. □

- Con pesi distinti, l'algoritmo di Kruskal, l'algoritmo di Prim da qualsiasi vertice di partenza e ogni esecuzione dell'algoritmo greedy generico restituiscono lo stesso albero.
- Sul grafo di riferimento, l'algoritmo di Prim a partire da $s = 6$ aggiunge gli spigoli $\{5,6\},\{4,5\},\{3,4\},\{1,3\},\{2,3\}$: ancora una volta l'albero $\tilde{F}$.

2.7.1 Pesi uguali

Proposizione 2.16: spigoli in tutti o in nessun albero ricoprente di costo minimo

Sia T un albero ricoprente di costo minimo di $G = (V, E)$.

(a) Uno spigolo $e \in T$ appartiene a ogni albero ricoprente di costo minimo se e solo se $w_e < w_f$ per ogni $f \neq e$ nel taglio fondamentale di e.

(b) Uno spigolo $e \notin T$ non appartiene a nessun albero ricoprente di costo minimo se e solo se $w_e > w_f$ per ogni $f \neq e$ in $C(T, e)$.

In particolare, T è l'unico albero ricoprente di costo minimo se e solo se ogni spigolo fuori da T è strettamente più pesante degli altri spigoli del suo ciclo fondamentale.

Dimostrazione. (a) Se le disuguaglianze valgono, e sta in ogni albero ricoprente di costo minimo per la proprietà del taglio (Teorema 2.6(b)). Altrimenti qualche $f \neq e$ nel taglio fondamentale ha $w_f \leq w_e$, e $w_f \geq w_e$ per la condizione del taglio (Teorema 2.8): quindi $w_f = w_e$, e $T \cup \{f\} \setminus \{e\}$ è un albero ricoprente (Lemma 2.5(2)) dello stesso peso, senza e.

(b) Se le disuguaglianze valgono, e non sta in nessun albero ricoprente di costo minimo per la proprietà del ciclo (Teorema 2.7(b)). Altrimenti qualche $f \neq e$ in $C(T, e)$ ha $w_f = w_e$, per la condizione del ciclo, e $T \cup \{e\} \setminus \{f\}$ è un albero ricoprente di costo minimo che contiene e.

Per l'ultima affermazione: T è unico se e solo se nessuno spigolo fuori da T appartiene a un albero ricoprente di costo minimo, cioè se (b) vale per ogni $e \notin T$. □

Esempio 2.9: pesi uguali sul grafo di riferimento

- Poniamo $w_{12} = 3$ invece di 4, in modo che $w_{12} = w_{23} = 3$. Il ciclo fondamentale di $\{1, 2\}$ in $\tilde{F}$ è $1, 3, 2, 1$, con $w_{23} = w_{12}$: per la Proposizione 2.16(b), anche l'albero $\{1, 2\}, \{1, 3\}, \{3, 4\}, \{4, 5\}, \{5, 6\}$ è ottimo, con peso 18.
- L'enumerazione conferma che questi sono gli unici due alberi ricoprenti di costo minimo. L'algoritmo di Kruskal, che esamina $\{1, 2\}$ prima di $\{2, 3\}$, restituisce quello con $\{1, 2\}$.
- I pesi uguali non creano sempre più alberi ottimi: con $w_{35} = w_{46} = 9$ l'albero $\tilde{F}$ è ancora l'unico albero ricoprente di costo minimo, perché ciascuno dei quattro spigoli fuori dall'albero resta strettamente il più pesante del suo ciclo.

Con pesi distinti l'albero ricoprente di costo minimo è unico. Con pesi uguali, un albero è l'unico ottimo se e solo se ogni spigolo fuori dall'albero è strettamente il più pesante del suo ciclo fondamentale.

2.8 Errori tipici

Attenzione

- **Usare la proprietà del taglio con un taglio attraversato da F.** Lo spigolo più leggero di $\delta(S)$ è sicuro solo se nessuno spigolo già scelto attraversa $\delta(S)$.
- **Accettare uno spigolo nell'algoritmo di Kruskal perché tocca un vertice nuovo.** Il test è se i due estremi stanno in componenti diverse: $\{3, 4\}$ è accettato anche se 3 e 4 sono già entrambi coperti.
- **Aggiornare le etichette nell'algoritmo di Prim con le lunghezze dei cammini.** La nuova etichetta è il peso w_{vu} di un solo spigolo, non $d(v) + w_{vu}$: quello è l'algoritmo di Dijkstra per i cammini minimi.
- **Dimenticare che le etichette possono solo diminuire.** Nell'algoritmo di Prim $d(u)$ cambia solo quando $w_{vu} < d(u)$.
- **Dedurre l'unicità dall'algoritmo.** Kruskal e Prim restituiscono un solo albero; con pesi uguali possono esistere altri alberi ottimi, e si trovano attraverso le uguaglianze nei cicli fondamentali.
- **Applicare gli algoritmi ai grafi orientati.** L'albero ricoprente di costo minimo è definito sui grafi non orientati; in un grafo orientato si ignora l'orientamento degli archi.

2.9 In sintesi

In sintesi

- Un albero ricoprente di costo minimo è un albero ricoprente di peso totale minimo di un grafo pesato connesso; ha $n - 1$ spigoli.
- Proprietà del taglio: lo spigolo più leggero di un taglio non attraversato da F estende F dentro qualche albero ricoprente di costo minimo. Proprietà del ciclo: lo spigolo più pesante di un ciclo si può lasciare fuori, e se è strettamente il più pesante non sta in nessun albero ottimo.
- Un albero ricoprente è ottimo se e solo se ogni spigolo fuori dall'albero è il più pesante del suo ciclo fondamentale, o equivalentemente ogni spigolo dell'albero è il più leggero del suo taglio fondamentale.
- L'algoritmo di Kruskal ordina gli spigoli e accetta quelli che uniscono due componenti, in tempo $O(m \log n)$; l'algoritmo di Prim fa crescere un albero con le etichette $d(v)$, in tempo $O(n^2)$.
- Con pesi distinti l'albero ricoprente di costo minimo è unico.

Il grafo di riferimento pesato in breve.

vertici, spigoli, alberi ricoprenti	$n = 6$, $m = 9$, 55
albero ricoprente di costo minimo	$\{1,3\}, \{2,3\}, \{3,4\}, \{4,5\}, \{5,6\}$, peso 18 (unico)
Kruskal	accetta $\{1,3\}, \{4,5\}, \{2,3\}, \{3,4\}, \{5,6\}$; rifiuta $\{1,2\}, \{2,4\}$
Prim da 1	seleziona $3, 2, 4, 5, 6$

Capitolo 3

Cammini minimi

Prerequisiti

- Dal Capitolo 1 «Grafi»: passeggiate, cammini e cicli orientati, la matrice di incidenza nodo-arco, gli alberi e il grafo di riferimento.
- Dal modulo Programmazione lineare, Capitolo 9 «Teoria della dualità»: il duale di un modello LP e il teorema della dualità debole.

Obiettivi

Alla fine di questo capitolo sarai in grado di:

- enunciare il problema del cammino minimo s–t e quello a sorgente singola, e riconoscere quando un ciclo negativo li rende mal posti;
- usare l'ottimalità dei sottocammini, le equazioni di Bellman e l'albero dei cammini minimi;
- scrivere il modello LP di un cammino minimo come flusso unitario, scriverne il duale con i potenziali π_i e usare potenziali ammissibili come certificato di ottimalità;
- eseguire l'algoritmo per i grafi aciclici, l'algoritmo di Bellman–Ford (con il rilevamento di un ciclo negativo) e l'algoritmo di Dijkstra, dimostrarne la correttezza e darne il tempo di esecuzione.

3.1 Problemi di cammino minimo

Definizione 3.1: problemi di cammino minimo

Sia $G = (N, A)$ un grafo orientato con una **lunghezza** $d_{ij} \in \mathbb{R}$ per ogni arco $(i, j) \in A$, di qualsiasi segno.

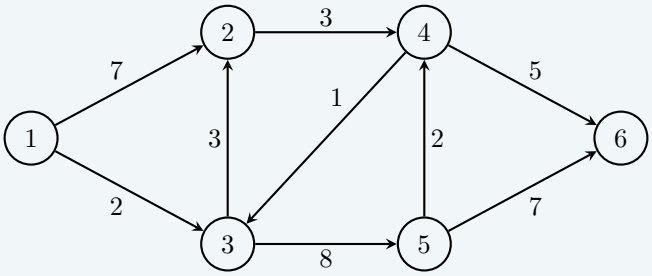
- La lunghezza di una passeggiata orientata $W = i_0, i_1, \ldots, i_k$ è la somma delle lunghezze dei suoi archi, $d(W) = \sum_{t=1}^{k} d_{i_{t-1} i_t}$.
- Dati due nodi s e t, il **problema del cammino minimo** s–t chiede un cammino orientato da s a t di lunghezza minima.
- Dato un nodo s, la **sorgente**, il **problema dei cammini minimi a sorgente singola** chiede un cammino orientato minimo da s a ogni nodo $j \in N$.

La lunghezza minima di un cammino orientato da s a j è la **distanza** $\tilde{d}(j)$ di j; poniamo $\tilde{d}(j) = +\infty$ se j non è raggiungibile da s.

- In questo capitolo scriviamo $n = |N|$, $m = |A|$, e "cammino" e "ciclo" indicano sempre un cammino orientato e un ciclo orientato.
- Un algoritmo per il problema a sorgente singola risolve anche il problema s–t, per tutti i t insieme.

Esempio 3.1: il grafo di riferimento con le lunghezze

Diamo al grafo di riferimento del Capitolo 1 «Grafi» le lunghezze d_{ij} scritte sugli archi, tutte non negative; la sorgente è $s = 1$.



Tre dei cammini da 1 a 6 e le loro lunghezze sono:

$d(1, 2, 4, 6) = 7{+}3{+}5 = 15, \qquad d(1, 3, 2, 4, 6) = 2{+}3{+}3{+}5 = 13, \qquad d(1, 3, 5, 6) = 2{+}8{+}7 = 17.$

Le distanze dal nodo 1, calcolate nella Sezione 3.6, sono

$$\tilde{d} = (0,\ 5,\ 2,\ 8,\ 10,\ 13),$$

quindi $1, 3, 2, 4, 6$ è un cammino minimo da 1 a 6.

3.1.1 Cicli negativi

Definizione 3.2: ciclo negativo

Un **ciclo negativo** è un ciclo orientato C con $d(C) < 0$. Nel resto del capitolo facciamo riferimento a due ipotesi:

(A1) ogni nodo di G è raggiungibile da s;

(A2) nessun ciclo negativo di G è raggiungibile da s.

Lemma 3.3: passeggiate e cammini

Sotto (A2), per ogni passeggiata orientata W da s a j esiste un cammino orientato P da s a j, con archi di W, tale che $d(P) \leq d(W)$.

Dimostrazione. Per induzione sul numero di archi di W. Se W ha nodi distinti, $P = W$.

1. Altrimenti W ripete un nodo: $W = s, \ldots, i_a, \ldots, i_b, \ldots, j$ con $i_a = i_b$, $a < b$ e $b - a$ minimo. Il tratto $C = i_a, \ldots, i_b$ è un ciclo, raggiungibile da s lungo W.
2. Per (A2), $d(C) \geq 0$. Togliendo C da W si ottiene una passeggiata W' da s a j con meno archi e $d(W') = d(W) - d(C) \leq d(W)$.
3. Per induzione W' contiene un cammino P con $d(P) \leq d(W') \leq d(W)$. □

- Sotto (A2) la lunghezza minima di una passeggiata da s a j coincide con la distanza $\tilde{d}(j)$: gli algoritmi di questo capitolo calcolano lunghezze di passeggiate, e il lemma le trasforma in cammini.
- Con un ciclo negativo C raggiungibile da s e dal quale j è raggiungibile, percorrendo C ancora e ancora si ottengono passeggiate da s a j di lunghezza negativa arbitrariamente grande in valore assoluto. Gli algoritmi di questo capitolo, allora, rilevano C invece di calcolare le distanze.

Esempio 3.2: un ciclo negativo

Il grafo di riferimento ha due cicli, $2, 4, 3, 2$ e $3, 5, 4, 3$. Con le lunghezze dell'Esempio 3.1 le loro lunghezze sono 7 e 11. Se la lunghezza dell'arco $(5, 4)$ scende da 2 a $d''_{54} = -10$, il ciclo $3, 5, 4, 3$ ha lunghezza

$$8 - 10 + 1 = -1,$$

e le passeggiate $1, 3, 5, 6,\ 1, 3, 5, 4, 3, 5, 6,\ 1, 3, 5, 4, 3, 5, 4, 3, 5, 6, \ldots$ da 1 a 6 hanno lunghezze $17, 16, 15, \ldots$: nessuna passeggiata è minima. Useremo di nuovo queste lunghezze d'' nella Sezione 3.5.

3.1.2 Ottimalità dei sottocammini

Lemma 3.4: ottimalità dei sottocammini

Sotto (A2), sia P un cammino minimo da s a j e sia i un nodo di P. Allora il sottocammino P_{si} di P da s a i è un cammino minimo da s a i.

Dimostrazione. Scriviamo P come P_{si} seguito dal sottocammino P_{ij} da i a j, quindi $d(P) = d(P_{si}) + d(P_{ij})$. Supponiamo per assurdo che un cammino Q da s a i abbia $d(Q) < d(P_{si})$.

1. Q seguito da P_{ij} è una passeggiata da s a j di lunghezza $d(Q) + d(P_{ij}) < d(P)$.
2. Per il Lemma 3.3 essa contiene un cammino da s a j più corto di P: assurdo. □

Teorema 3.5: equazioni di Bellman

Sotto (A1) e (A2), le distanze soddisfano

$$\tilde{d}(s) = 0, \qquad \tilde{d}(j) = \min_{(i,j)\in\delta^-(j)} \{\, \tilde{d}(i) + d_{ij} \,\}, \quad \forall j \in N \setminus \{s\},$$

e $\tilde{d}(j) \le \tilde{d}(i) + d_{ij}$ per ogni arco $(i,j) \in A$.

Dimostrazione. Tutte le distanze sono finite per (A1), e $\tilde{d}(s) = 0$: il cammino s ha lunghezza 0, e un cammino da s a s con archi sarebbe un ciclo, di lunghezza ≥ 0 per (A2).

- *Disuguaglianza.* Un cammino minimo da s a i seguito dall'arco (i,j) è una passeggiata di lunghezza $\tilde{d}(i) + d_{ij}$; per il Lemma 3.3, $\tilde{d}(j) \le \tilde{d}(i) + d_{ij}$.
- *Equazione.* Per $j \neq s$, sia P un cammino minimo da s a j e (i,j) il suo ultimo arco. Per il Lemma 3.4, $\tilde{d}(j) = d(P) = \tilde{d}(i) + d_{ij}$, quindi il minimo è raggiunto. □

Proposizione 3.6: albero dei cammini minimi

Sotto (A1) e (A2), esiste un albero ricoprente T di G, l'**albero dei cammini minimi**, tale che per ogni $j \in N$ il cammino da s a j in T è orientato ed è un cammino minimo.

Dimostrazione. Per ogni j, sia $k(j)$ il numero minimo di archi di un cammino minimo da s a j; $k(s) = 0$.

1. Per $j \neq s$, scegliamo un cammino minimo P da s a j con $k(j)$ archi, e sia $pred(j)$ il nodo che precede j su P.
2. Il sottocammino di P da s a $pred(j)$ è minimo (Lemma 3.4) e ha $k(j) - 1$ archi, quindi $k(pred(j)) \le k(j) - 1$ e $\tilde{d}(j) = \tilde{d}(pred(j)) + d_{pred(j)\,j}$.
3. Passando da j a $pred(j)$ il numero k diminuisce strettamente, quindi la sequenza

termina in s. Gli archi $(pred(j), j)$ sono $n-1$ e collegano ogni nodo a s: formano un albero ricoprente (Capitolo 1, Teorema «caratterizzazioni di un albero»).

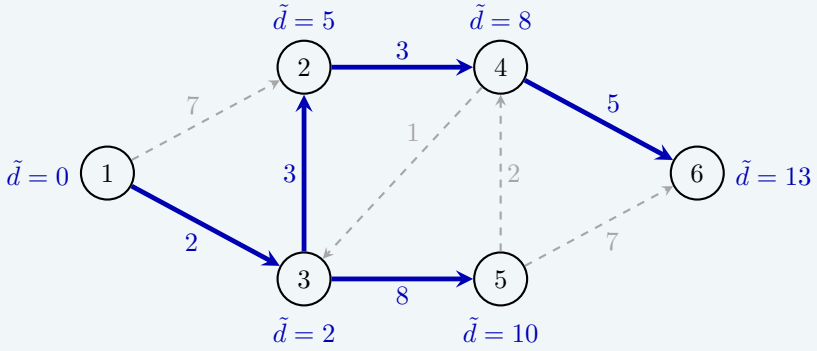
4. Lungo il cammino dell'albero $s = i_0, i_1, \ldots, i_r = j$ ogni arco soddisfa $\tilde{d}(i_t) = \tilde{d}(i_{t-1}) + d_{i_{t-1}i_t}$; sommando, la sua lunghezza è $\tilde{d}(j)$. □

Esempio 3.3: le equazioni di Bellman sul grafo di riferimento

Con le distanze $\tilde{d} = (0, 5, 2, 8, 10, 13)$ dell'Esempio 3.1:

- il nodo 4 ha gli archi entranti $(2,4)$ e $(5,4)$, e $\tilde{d}(4) = \min\{5+3,\ 10+2\} = 8$;
- il nodo 6 ha gli archi entranti $(4,6)$ e $(5,6)$, e $\tilde{d}(6) = \min\{8+5,\ 10+7\} = 13$.

Gli archi che realizzano il minimo formano l'albero dei cammini minimi $(1,3)$, $(3,2)$, $(2,4)$, $(3,5)$, $(4,6)$, in blu; accanto a ogni nodo la sua distanza.



Senza cicli negativi raggiungibili dalla sorgente, ogni sottocammino di un cammino minimo è minimo, le distanze soddisfano le equazioni di Bellman, e un solo albero ricoprente contiene un cammino minimo verso ogni nodo.

3.2 Condizioni di ottimalità e potenziali

Definizione 3.7: potenziali e costi ridotti

Un vettore $\boldsymbol{\pi} \in \mathbb{R}^n$ assegna un **potenziale** π_i a ogni nodo $i \in N$. Il **costo ridotto** dell'arco (i,j) rispetto a $\boldsymbol{\pi}$ è

$$\bar{c}_{ij} = d_{ij} - \pi_i + \pi_j.$$

I potenziali sono **ammissibili** se $\bar{c}_{ij} \geq 0$ per ogni arco $(i,j) \in A$, e un arco con $\bar{c}_{ij} = 0$ è **stretto**.

- Date delle **etichette** $d(j)$, lunghezze di passeggiate da s a j, usiamo i potenziali $\pi_i = -d(i)$. Allora $\bar{c}_{ij} = d_{ij} + d(i) - d(j)$, e i potenziali sono ammissibili se e solo se $d(j) \leq d(i) + d_{ij}$ per ogni arco.

- Lungo una passeggiata $W = i_0, i_1, \dots, i_k$ i potenziali si elidono a catena (somma telescopica):

$$d(W) = \sum_{t=1}^{k} \bar{c}_{i_{t-1} i_t} \; + \; \pi_{i_0} - \pi_{i_k}.$$

Teorema 3.8: potenziali come certificato di ottimalità

Siano $\boldsymbol{\pi}$ potenziali ammissibili e sia P un cammino da s a t i cui archi sono tutti stretti. Allora P è un cammino minimo da s a t, e $d(P) = \pi_s - \pi_t$. Inoltre G non ha cicli negativi.

Dimostrazione. Usiamo la formula telescopica qui sopra.

1. Gli archi di P hanno $\bar{c}_{ij} = 0$, quindi $d(P) = \pi_s - \pi_t$.
2. Per ogni cammino Q da s a t, $d(Q) = \sum_{(i,j)\in Q} \bar{c}_{ij} + \pi_s - \pi_t \geq \pi_s - \pi_t = d(P)$.
3. Per ogni ciclo C il primo e l'ultimo nodo coincidono, quindi $d(C) = \sum_{(i,j)\in C} \bar{c}_{ij} \geq 0$. □

Corollario 3.9: condizioni di ottimalità per le etichette

Siano le etichette $d(j)$, $j \in N$, finite con $d(s) = 0$, e supponiamo che

(i) $d(j) \leq d(i) + d_{ij}$ per ogni arco $(i, j) \in A$;

(ii) per ogni $j \neq s$ esista un cammino da s a j di lunghezza $d(j)$.

Allora $d(j) = \tilde{d}(j)$ per ogni $j \in N$.

Dimostrazione. Per (i) i potenziali $\pi_i = -d(i)$ sono ammissibili, quindi ogni cammino Q da s a j ha $d(Q) \geq \pi_s - \pi_j = d(j)$ con il ragionamento del Teorema 3.8. Per (ii) il valore $d(j)$ è realizzato da un cammino, quindi $d(j) = \tilde{d}(j)$. □

Esempio 3.4: un certificato sul grafo di riferimento

Con le distanze dell'Esempio 3.1 i potenziali sono $\boldsymbol{\pi} = -\tilde{\boldsymbol{d}} = (0, -5, -2, -8, -10, -13)$, e i costi ridotti $\bar{c}_{ij} = d_{ij} - \pi_i + \pi_j$ sono:

arco	$(1,2)$	$(1,3)$	$(2,4)$	$(3,2)$	$(3,5)$	$(4,3)$	$(4,6)$	$(5,4)$	$(5,6)$
d_{ij}	7	2	3	3	8	1	5	2	7
$\bar{c}_{ij}$	2	0	0	0	0	7	0	4	4

- Tutti i costi ridotti sono non negativi, e gli archi stretti sono gli archi dell'albero dei cammini minimi. Il cammino $1, 3, 2, 4, 6$ è stretto, quindi è minimo, con lunghezza $\pi_1 - \pi_6 = 13$.
- L'albero della visita in ampiezza, $(1,2)$, $(1,3)$, $(2,4)$, $(3,5)$, $(4,6)$, dà le etichette $(0, 7, 2, 10, 10, 15)$. Con i loro potenziali, $\bar{c}_{32} = 3 - (-2) + (-7) = -2 < 0$: i potenziali

non sono ammissibili, ed entrare nel nodo 2 attraverso $(3, 2)$ accorcia il suo cammino di 2.

Potenziali ammissibili ($\bar{c}_{ij} = d_{ij} - \pi_i + \pi_j \geq 0$ su ogni arco) e un cammino fatto di archi stretti dimostrano che il cammino è minimo. Con le etichette, $\pi_i = -d(i)$, e l'ammissibilità si legge $d(j) \leq d(i) + d_{ij}$.

3.3 La formulazione di programmazione lineare

- Un cammino da s a t invia un'unità di flusso da s a t. Usiamo una variabile $x_{ij} \geq 0$ per ogni arco, il flusso sull'arco, e i **bilanci**

$$b_s = 1, \qquad b_t = -1, \qquad b_i = 0, \quad \forall i \in N \setminus \{s, t\}.$$

Usando queste variabili, un modello LP per il problema del cammino minimo s–t è il seguente:

$$\min \sum_{(i,j)\in A} d_{ij}\, x_{ij} \tag{3.1a}$$

$$\text{soggetto a} \sum_{(i,j)\in\delta^+(i)} x_{ij} - \sum_{(j,i)\in\delta^-(i)} x_{ji} = b_i, \qquad \forall i \in N, \tag{3.1b}$$

$$x_{ij} \geq 0, \qquad \forall (i,j) \in A. \tag{3.1c}$$

Questo modello LP si indica come modello $\mathrm{SP}(s, t)$.

- Descrizione della funzione obiettivo e dei vincoli del modello $\mathrm{SP}(s, t)$:
  - la funzione obiettivo (3.1a) minimizza la lunghezza totale del flusso;
  - i vincoli (3.1b) sono i vincoli di conservazione del flusso: un'unità esce da s, un'unità entra in t, e ogni altro nodo invia quello che riceve;
  - infine, i vincoli (3.1c) definiscono le variabili del modello LP.
- In forma matriciale il modello è $\min\{\boldsymbol{d}'\boldsymbol{x} : \boldsymbol{B}\boldsymbol{x} = \boldsymbol{b},\ \boldsymbol{x} \geq \boldsymbol{0}\}$, con la matrice di incidenza nodo-arco $\boldsymbol{B}$ del Capitolo 1: $+1$ nella coda e -1 nella testa di ogni arco.
- Il **vettore di incidenza** $\boldsymbol{x}^P$ di un cammino P da s a t, con $x^P_{ij} = 1$ sugli archi di P e 0 altrove, è ammissibile, e il suo valore è $d(P)$.

3.3.1 Il modello duale e i potenziali

- Il vincolo del nodo i è un'equazione, quindi la sua variabile duale π_i è libera. La colonna di x_{ij} in $\boldsymbol{B}$ ha $+1$ nella riga i e -1 nella riga j, quindi il vincolo duale di $x_{ij} \geq 0$ è $\pi_i - \pi_j \leq d_{ij}$ (Programmazione lineare, Capitolo 9 «Teoria della dualità»).

Usando una variabile duale libera π_i per ciascun vincolo (3.1b), il modello duale del modello $\mathrm{SP}(s,t)$ è:

$$\max \quad \pi_s - \pi_t \tag{3.2a}$$

$$\text{soggetto a} \quad \pi_i - \pi_j \leq d_{ij}, \qquad \forall (i,j) \in A, \tag{3.2b}$$

$$\pi_i \in \mathbb{R}, \qquad \forall i \in N. \tag{3.2c}$$

Questo modello LP si indica come modello $D(\mathrm{SP}(s,t))$.

- Il vincolo (3.2b) è $\bar{c}_{ij} = d_{ij} - \pi_i + \pi_j \geq 0$: le soluzioni ammissibili del duale sono i potenziali ammissibili della Definizione 3.7, e $\bar{c}_{ij}$ è il costo ridotto di x_{ij} nel senso della programmazione lineare.

Teorema 3.10: LP e cammini minimi

Sotto (A1) e (A2), il modello $\mathrm{SP}(s,t)$ ha una soluzione ottima e $z(\mathrm{SP}(s,t)) = \tilde{d}(t)$. Il vettore di incidenza di un cammino minimo da s a t è ottimo per $\mathrm{SP}(s,t)$, e i potenziali $\pi_i = -\tilde{d}(i)$ sono ottimi per $D(\mathrm{SP}(s,t))$.

Dimostrazione. Sia P un cammino minimo da s a t, che esiste per (A1).

1. $\boldsymbol{x}^P$ è ammissibile per $\mathrm{SP}(s,t)$, con valore $d(P) = \tilde{d}(t)$.
2. Per il Teorema 3.5, $\tilde{d}(j) \leq \tilde{d}(i) + d_{ij}$ per ogni arco, quindi $\pi_i = -\tilde{d}(i)$ soddisfa $\pi_i - \pi_j \leq d_{ij}$; il suo valore è $\pi_s - \pi_t = 0 + \tilde{d}(t)$.
3. Una soluzione primale ammissibile e una soluzione duale ammissibile con lo stesso valore sono entrambe ottime, per la dualità debole. □

Proposizione 3.11: un ciclo negativo rende il modello illimitato

Se $\mathrm{SP}(s,t)$ è ammissibile e G ha un ciclo negativo C, raggiungibile da s o no, allora $\mathrm{SP}(s,t)$ è illimitato.

Dimostrazione. Sia $\bar{\boldsymbol{x}}$ ammissibile e sia $\boldsymbol{x}^C$ il vettore di incidenza di C. Le colonne degli archi di C sommano a zero (Capitolo 1, Lemma «colonne di un ciclo»), quindi $\boldsymbol{B}\boldsymbol{x}^C = \boldsymbol{0}$. Per ogni $\lambda \geq 0$ il vettore $\bar{\boldsymbol{x}} + \lambda \boldsymbol{x}^C$ è ammissibile, con valore $\boldsymbol{d}'\bar{\boldsymbol{x}} + \lambda\, d(C)$, che tende a $-\infty$. □

Il segno dei potenziali

Con la matrice di incidenza del corso (+1 nella coda) il vincolo duale è $\pi_i - \pi_j \le d_{ij}$, e i potenziali ottimi sono $\pi_i = -\tilde{d}(i)$, a meno di una costante additiva. La forma $\pi_j - \pi_i \le d_{ij}$, con $\pi_i = \tilde{d}(i)$, appartiene alla convenzione di segno opposta (-1 nella coda).

3.3.2 Il modello a sorgente singola

- Per trovare un cammino minimo da s a ogni nodo, s invia un'unità a ciascuno degli altri $n-1$ nodi:
$$b_s = n-1, \qquad b_i = -1, \quad \forall i \in N \setminus \{s\}.$$
Con questi bilanci il modello (3.1) si chiama SP(s); il suo duale massimizza $(n-1)\pi_s - \sum_{i \neq s} \pi_i$ con gli stessi vincoli (3.2b).

Proposizione 3.12: il modello a sorgente singola

Sotto (A1) e (A2), sia T un albero dei cammini minimi e, per ogni $j \neq s$, sia T_j l'insieme dei nodi il cui cammino nell'albero passa per j (j compreso). Il flusso con $x_{ij} = |T_j|$ sugli archi (i,j) di T e 0 altrove è ottimo per SP(s), i potenziali $\pi_i = -\tilde{d}(i)$ sono ottimi per il suo duale, e
$$z(\mathrm{SP}(s)) = \sum_{j \in N} \tilde{d}(j).$$

Dimostrazione. 1. *Ammissibilità.* In un nodo $j \neq s$ il flusso che entra sull'arco dell'albero è $|T_j|$, e il flusso che esce è $\sum |T_h|$ sugli archi dell'albero (j,h), cioè $|T_j| - 1$: il bilancio è -1. In s il flusso uscente copre tutti gli altri nodi: $n-1$.

2. *Valore.* Il flusso è la somma dei vettori di incidenza degli $n-1$ cammini dell'albero, perché l'arco (i,j) sta sui cammini dell'albero dei $|T_j|$ nodi di T_j. Il suo valore è $\sum_j \tilde{d}(j)$.

3. *Duale.* $\pi_i = -\tilde{d}(i)$ è ammissibile come nel Teorema 3.10, con valore $(n-1)\cdot 0 + \sum_{i \neq s} \tilde{d}(i)$. Per la dualità debole entrambe le soluzioni sono ottime. □

- Nel Capitolo 4 «Modelli di flusso su reti» si mostra che i vertici di questi poliedri sono interi, per la totale unimodularità di $\boldsymbol{B}$.

Esempio 3.5: i modelli LP sul grafo di riferimento

Risolviamo i modelli con le lunghezze dell'Esempio 3.1, $s = 1$ e $t = 6$ (valori verificati con Gurobi).

- SP$(1,6)$: $z(\mathrm{SP}(1,6)) = 13$, con $x_{13} = x_{32} = x_{24} = x_{46} = 1$ e tutte le altre variabili a 0: il cammino $1,3,2,4,6$. Il valore duale è $\pi_1 - \pi_6 = 13$.
- SP(1): $z(\mathrm{SP}(1)) = 0+5+2+8+10+13 = 38$, con i flussi sull'albero dei cammini minimi
$$x_{13} = 5, \qquad x_{32} = 3, \qquad x_{24} = 2, \qquad x_{46} = 1, \qquad x_{35} = 1.$$

Per esempio $T_3 = \{2, 3, 4, 5, 6\}$, quindi $x_{13} = 5$; il costo è $2 \cdot 5 + 3 \cdot 3 + 3 \cdot 2 + 5 + 8 = 38$. I duali ottimi, normalizzati con $\pi_1 = 0$, sono $\boldsymbol{\pi} = (0, -5, -2, -8, -10, -13)$.

- Con le lunghezze d'' dell'Esempio 3.2 il modello $\mathrm{SP}(1, 6)$ è illimitato: ogni unità di flusso lungo il ciclo $3, 5, 4, 3$ abbassa il valore di 1.

Un cammino minimo è un flusso unitario da s a t: $\min \boldsymbol{d}'\boldsymbol{x}$ con $\boldsymbol{B}\boldsymbol{x} = \boldsymbol{b}$, $\boldsymbol{x} \geq \boldsymbol{0}$. Il suo duale ha i vincoli $\pi_i - \pi_j \leq d_{ij}$, e $\pi_i = -\tilde{d}(i)$ è ottimo; un ciclo negativo in un punto qualsiasi di G rende il modello illimitato.

3.4 Cammini minimi nei grafi aciclici

Definizione 3.13: grafo aciclico e ordine topologico

Un grafo orientato senza cicli orientati è un **grafo aciclico orientato** (DAG). Un **ordine topologico** di $G = (N, A)$ è un ordinamento $v_1, v_2, \ldots, v_n$ dei suoi nodi tale che ogni arco va in avanti:

$$(v_a, v_b) \in A \quad \Longrightarrow \quad a < b.$$

Proposizione 3.14: ordini topologici

Un grafo orientato ha un ordine topologico se e solo se è aciclico.

Dimostrazione. Se G ha un ordine topologico, un ciclo $i_0, i_1, \ldots, i_k = i_0$ andrebbe in avanti su ogni arco e tornerebbe al suo primo nodo: impossibile. Viceversa, sia G aciclico; ragioniamo per induzione su n.

1. G ha un nodo senza archi entranti: altrimenti, camminando all'indietro lungo archi entranti a partire da un nodo qualsiasi, dopo n passi un nodo si ripete, e la passeggiata contiene un ciclo.
2. Mettiamo un nodo di questo tipo v_1 per primo. Il grafo senza v_1 è aciclico, e per induzione ha un ordine topologico $v_2, \ldots, v_n$; nessun arco entra in v_1, quindi $v_1, v_2, \ldots, v_n$ è un ordine topologico di G. □

Algoritmo

Algorithm: Cammini minimi in un grafo aciclico

Input: un grafo aciclico $G = (N, A)$ con lunghezze d_{ij} di qualsiasi segno, una sorgente $s \in N$

Output: le distanze $d(j) = \tilde{d}(j)$ e i predecessori $pred(j)$

1 **Passo 0 (ordine).** Trovare un ordine topologico: togliere ripetutamente un nodo senza archi entranti (quello di indice minimo, se ce n'è più di uno).
2 Porre $d(s) := 0$ e $d(j) := +\infty$ per $j \neq s$.
3 **Passo 1 (etichette).** Per ogni nodo i in ordine topologico con $d(i) < +\infty$, e per ogni arco $(i, j) \in \delta^+(i)$:
4 se $d(i) + d_{ij} < d(j)$, porre $d(j) := d(i) + d_{ij}$ e $pred(j) := i$.
5 `STOP`.

Teorema 3.15: correttezza e tempo di esecuzione sui grafi aciclici

Su un grafo aciclico con lunghezze di qualsiasi segno, l'algoritmo «Cammini minimi in un grafo aciclico» termina con $d(j) = \tilde{d}(j)$ per ogni $j \in N$, e richiede tempo $O(n + m)$.

Dimostrazione. Un grafo aciclico non ha alcun ciclo, quindi vale (A2). Ogni etichetta finita è la lunghezza di una passeggiata da s, quindi $d(j) \geq \tilde{d}(j)$ per il Lemma 3.3; le etichette possono solo diminuire.

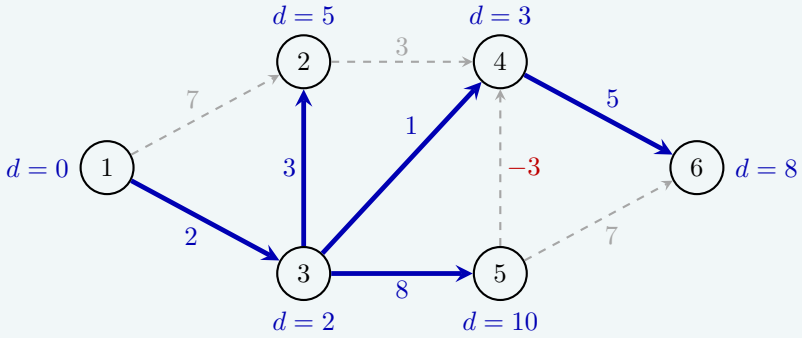
1. Sia j raggiungibile e $s = i_0, i_1, \ldots, i_k = j$ un cammino minimo; i suoi nodi compaiono in quest'ordine nell'ordine topologico, perché ogni arco va in avanti.
2. Per induzione su t, dopo l'elaborazione di i_t vale $d(i_t) \leq \tilde{d}(i_t)$: il cammino fino a i_{t-1} è minimo (Lemma 3.4), e l'elaborazione di i_{t-1} dà $d(i_t) \leq \tilde{d}(i_{t-1}) + d_{i_{t-1}i_t} = \tilde{d}(i_t)$.
3. I nodi non raggiungibili da s conservano $d(j) = +\infty = \tilde{d}(j)$.

Tempo. L'ordine topologico richiede tempo $O(n + m)$ con un contatore degli archi entranti di ogni nodo, decrementato quando si toglie una coda; il Passo 1 esamina ogni arco una volta. □

- La regola dell'indice minimo fissa un ordine per gli esempi; si può togliere qualsiasi nodo senza archi entranti.
- Con le lunghezze $-d_{ij}$ l'algoritmo calcola i **cammini massimi**, perché un grafo aciclico non ha cicli da ripetere. I cammini massimi nei grafi aciclici sono il cuore del Capitolo 9 «Gestione dei progetti».

Esempio 3.6: un'istanza aciclica

Invertendo l'arco $(4, 3)$ del grafo di riferimento in $(3, 4)$, con $d_{34} = 1$, si ottiene un grafo aciclico. Gli diamo le lunghezze dell'Esempio 3.1, tranne $d_{54} = -3$ (in rosso).



Il Passo 0 dà l'ordine topologico $1, 3, 2, 5, 4, 6$: dopo 1, solo 3 non ha archi entranti; poi 2 e 5 sono liberi, e 2 va per primo. Ogni riga della tabella elabora un nodo e mostra le etichette dopo l'elaborazione:

passo	nodo i	$d(1)$	$d(2)$	$d(3)$	$d(4)$	$d(5)$	$d(6)$
1	1	0	7	2	∞	∞	∞
2	3	0	5	2	3	10	∞
3	2	0	5	2	3	10	∞
4	5	0	5	2	3	10	17
5	4	0	5	2	3	10	8
6	6	0	5	2	3	10	8

I predecessori danno l'albero $(1,3)$, $(3,2)$, $(3,4)$, $(3,5)$, $(4,6)$ (in blu). L'arco negativo $(5,4)$ non viene usato: $d(5) - 3 = 7 > 3 = d(4)$. Con le lunghezze $-d_{ij}$ le lunghezze dei cammini massimi da 1 sono $(0, 7, 2, 10, 10, 17)$.

Un grafo è aciclico se e solo se ha un ordine topologico. Esaminando i nodi in quest'ordine, ogni etichetta è definitiva quando il suo nodo viene elaborato: cammini minimi in tempo $O(n+m)$, con lunghezze di qualsiasi segno.

3.5 L'algoritmo di Bellman–Ford

Definizione 3.16: etichette con al più k archi

Per $k \geq 0$ e $j \in N$, $d^k(j)$ è la lunghezza minima di una passeggiata da s a j con al più k archi, e $d^k(j) = +\infty$ se non ce n'è nessuna.

Lemma 3.17: ricorsione delle etichette

$d^0(s) = 0$, $d^0(j) = +\infty$ per $j \neq s$, e per ogni $k \geq 1$

$$d^k(j) = \min\left\{ d^{k-1}(j),\ \min_{(i,j)\in\delta^-(j)} \left(d^{k-1}(i) + d_{ij}\right)\right\}, \qquad \forall j \in N.$$

Dimostrazione. L'unica passeggiata con 0 archi è la passeggiata s. Per $k \geq 1$, una passeggiata verso j con al più k archi ha al più $k-1$ archi, con lunghezza minima $d^{k-1}(j)$, oppure ha esattamente k archi e ultimo arco (i,j): è una passeggiata verso i con $k-1$ archi seguita da (i,j), e la sua lunghezza minima è $\min_{(i,j)} \left(d^{k-1}(i)+d_{ij}\right)$. □

Algoritmo

Algorithm: Algoritmo di Bellman–Ford

Input: un grafo orientato $G=(N,A)$ con lunghezze d_{ij} di qualsiasi segno, una sorgente $s \in N$

Output: le distanze $\tilde{d}(j)$ e i predecessori, oppure un ciclo negativo raggiungibile da s

1 **Passo 0 (inizializzazione).** Porre $d^0(s) := 0$, $d^0(j) := +\infty$ per $j \neq s$, e $k := 1$.
2 **Passo 1 (passata k).** Calcolare $d^k(j)$ per ogni $j \in N$ con la ricorsione del Lemma 3.17;
3 se $d^k(j) < d^{k-1}(j)$, porre $pred(j)$ uguale alla coda i dell'arco che realizza il minimo.
4 **Passo 2 (test).** Se $d^k = d^{k-1}$, `STOP`: $\tilde{d} = d^k$.
5 Se $k = n$, `STOP`: un ciclo negativo è raggiungibile da s. Altrimenti porre $k := k+1$ e andare al Passo 1.

- Ogni passata dipende solo dalle etichette della passata precedente: se $d^k = d^{k-1}$, tutte le passate successive ripetono le stesse etichette, e l'algoritmo può fermarsi.
- Negli esempi, i pareggi nel minimo si risolvono con la coda i di indice minimo, e il predecessore cambia solo quando l'etichetta diminuisce strettamente.

Teorema 3.18: correttezza di Bellman–Ford

Se nessun ciclo negativo è raggiungibile da s, allora $d^{n-1}(j) = \tilde{d}(j)$ per ogni $j \in N$, e l'algoritmo si ferma a una passata $k \leq n$ con $d^k = d^{k-1} = \tilde{d}$.

Dimostrazione. Un cammino ha al più $n-1$ archi, quindi $d^{n-1}(j) \leq \tilde{d}(j)$. Per il Lemma 3.3 (che richiede solo i cicli raggiungibili da s) ogni passeggiata da s a j contiene un cammino non più lungo, quindi $d^k(j) \geq \tilde{d}(j)$ per ogni k.

- Quindi $d^{n-1} = d^n = \tilde{d}$.
- Le etichette non aumentano mai, e l'algoritmo si ferma alla prima passata k con $d^k = d^{k-1}$, che è al più n; da quella passata in poi le etichette non cambiano più, quindi $d^k = d^{n-1} = \tilde{d}$. □

Teorema 3.19: rilevamento di un ciclo negativo

Un ciclo negativo è raggiungibile da s se e solo se $d^n \neq d^{n-1}$. In tal caso, sia $d^n(j) < d^{n-1}(j)$; leggendo i predecessori all'indietro a partire da j, una riga per passata, partendo

dalla riga n della tabella, si ottiene una passeggiata da s a j con n archi che contiene un ciclo negativo.

Dimostrazione. Se nessun ciclo negativo è raggiungibile da s, allora $d^n = d^{n-1}$ per il Teorema 3.18. Viceversa, sia $d^n(j) < d^{n-1}(j)$.

1. Ogni passeggiata verso j di lunghezza $d^n(j)$ con al più n archi ha esattamente n archi, altrimenti $d^{n-1}(j) \leq d^n(j)$.
2. Sia i il predecessore di j nella riga n, quindi $d^n(j) = d^{n-1}(i)+d_{ij}$. Una passeggiata che realizza $d^{n-1}(i)$ seguita da (i, j) realizza $d^n(j)$, quindi ha esattamente $n-1$ archi: $d^{n-1}(i) < d^{n-2}(i)$, e i ha un predecessore nella riga $n-1$.
3. Ripetendo il ragionamento fino alla riga 1, il cui predecessore è s, si ottiene una passeggiata W da s a j con n archi e $d(W) = d^n(j)$.
4. W ha $n+1$ nodi, quindi ripete un nodo e contiene un ciclo C. Togliendo C si ottiene una passeggiata verso j con meno di n archi, di lunghezza $d(W) - d(C) \geq d^{n-1}(j) > d^n(j) = d(W)$. Quindi $d(C) < 0$, e C è raggiungibile da s lungo W. □

Proposizione 3.20: l'albero dei predecessori di Bellman–Ford

Se nessun ciclo negativo è raggiungibile da s, all'arresto gli archi $(pred(j), j)$, per i nodi $j \neq s$ con etichetta finita, formano un albero dei cammini minimi dei nodi raggiungibili.

Dimostrazione. Siano $d = \tilde{d}$ le etichette finali, e sia $t(j)$ l'ultima passata in cui l'etichetta di j è diminuita; $t(s) = 0$, perché $d^k(s) = 0$ per ogni k. Sia $p = pred(j)$.

1. Nella passata $t(j)$, $d(j) = d^{t(j)-1}(p) + d_{pj} \geq d(p) + d_{pj}$, perché le etichette non aumentano mai.
2. Per il Teorema 3.5, $d(j) \leq d(p)+d_{pj}$. Quindi $d(j) = d(p)+d_{pj}$ e $d(p) = d^{t(j)-1}(p)$: l'etichetta di p non è cambiata dopo la passata $t(j)-1$, quindi $t(p) < t(j)$.
3. Passando al predecessore, t diminuisce strettamente, quindi la sequenza termina in s, lungo archi stretti: il cammino dell'albero da s a j ha lunghezza $d(j)$. □

Teorema 3.21: tempo di esecuzione di Bellman–Ford

L'algoritmo di Bellman–Ford richiede tempo $O(n\,m)$.

Dimostrazione. Una passata calcola $d^k(j)$ per ogni j esaminando la stella entrante $\delta^-(j)$, quindi esamina ogni arco una volta: tempo $O(n+m)$. Le passate sono al più n, e il totale è $O(n(n+m))$, cioè $O(n\,m)$ quando $m \geq n-1$, come sotto (A1). □

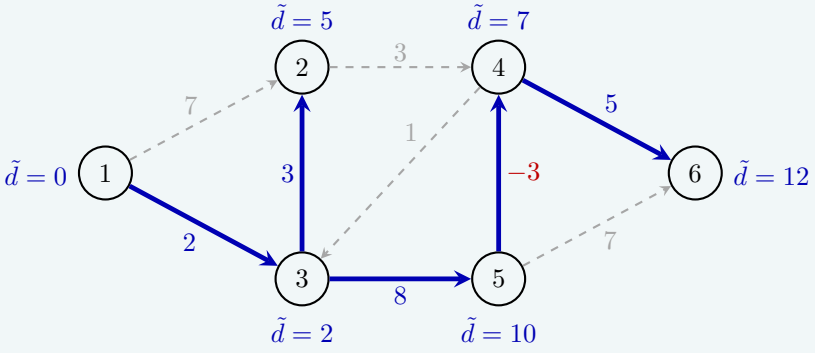
3.5.1 Bellman–Ford sul grafo di riferimento

Esempio 3.7: Bellman–Ford con un arco negativo

Usiamo le lunghezze d' dell'Esempio 3.1 con $d'_{54} = -3$ al posto di 2. I due cicli hanno lunghezze 7 e $8-3+1=6$, quindi non c'è alcun ciclo negativo. Ogni riga è una passata k; accanto a un'etichetta che è diminuita, tra parentesi il nuovo predecessore:

k	$d^k(1)$	$d^k(2)$	$d^k(3)$	$d^k(4)$	$d^k(5)$	$d^k(6)$
0	0	∞	∞	∞	∞	∞
1	0	7 (1)	2 (1)	∞	∞	∞
2	0	5 (3)	2	10 (2)	10 (3)	∞
3	0	5	2	7 (5)	10	15 (4)
4	0	5	2	7	10	12 (4)
5	0	5	2	7	10	12

- Nella passata 3, $d^3(4) = \min\{10,\ d^2(2)+3,\ d^2(5)-3\} = \min\{10, 8, 7\} = 7$, con predecessore 5.
- La passata 5 ripete le etichette della passata 4: l'algoritmo si ferma con $\tilde{d} = (0, 5, 2, 7, 10, 12)$, dopo 5 passate invece delle $n = 6$ consentite.
- L'albero dei cammini minimi ha gli archi $(1,3)$, $(3,2)$, $(3,5)$, $(5,4)$, $(4,6)$, e il cammino minimo verso 6 è $1, 3, 5, 4, 6$, di lunghezza $2+8-3+5=12$.



Esempio 3.8: Bellman–Ford rileva un ciclo negativo

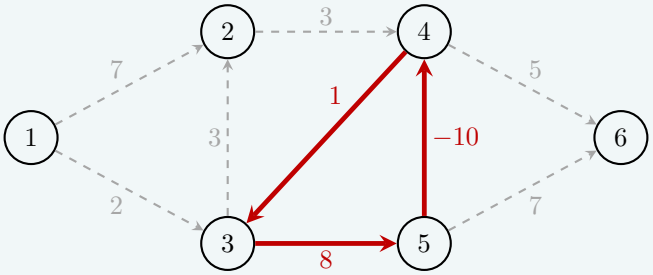
Con le lunghezze d'' dell'Esempio 3.2 ($d''_{54} = -10$) l'algoritmo esegue tutte le $n = 6$ passate:

k	$d^k(1)$	$d^k(2)$	$d^k(3)$	$d^k(4)$	$d^k(5)$	$d^k(6)$
0	0	∞	∞	∞	∞	∞
1	0	7 (1)	**2** (**1**)	∞	∞	∞
2	0	5 (3)	2	10 (2)	**10** (**3**)	∞
3	0	5	2	**0** (**5**)	10	15 (4)
4	0	5	**1** (**4**)	0	10	5 (4)
5	0	4 (3)	1	0	**9** (**3**)	5
6	0	4	1	**−1** (**5**)	9	5

- Nella passata 6 l'etichetta del nodo 4 scende da 0 a -1: $d^6 \neq d^5$, e un ciclo negativo è raggiungibile da 1.
- Leggendo all'indietro a partire dal nodo 4 (valori in grassetto): la riga 6 dà $pred = 5$, la riga 5 per il nodo 5 dà 3, la riga 4 per il nodo 3 dà 4, poi 5, 3, 1. La passeggiata è

  $$1,\ 3,\ 5,\ 4,\ 3,\ 5,\ 4, \qquad \text{di lunghezza } 2+8-10+1+8-10=-1=d^6(4).$$

- Il nodo 3 si ripete: la passeggiata contiene il ciclo $3, 5, 4, 3$, di lunghezza -1 (in rosso).



L'algoritmo di Bellman–Ford calcola d^k, le passeggiate minime con al più k archi, in tempo $O(n\,m)$. Senza cicli negativi raggiungibili da s si ferma entro n passate con le distanze; un cambiamento nella passata n rivela un ciclo negativo, che si trova leggendo i predecessori all'indietro.

3.6 L'algoritmo di Dijkstra

- In questa sezione tutte le lunghezze sono non negative: $d_{ij} \geq 0$ per ogni arco. Allora ogni ciclo ha lunghezza ≥ 0, e vale (A2).
- I nodi sono **permanenti**, quando la loro etichetta è definitiva, oppure **temporanei**. A ogni iterazione il nodo temporaneo con l'etichetta minima diventa permanente.

Algoritmo

Algorithm: Algoritmo di Dijkstra

Input: un grafo orientato $G = (N, A)$ con lunghezze $d_{ij} \geq 0$, una sorgente $s \in N$
Output: le distanze $d(j) = \tilde{d}(j)$ e i predecessori $pred(j)$

1 **Passo 0 (inizializzazione).** Porre $d(s) := 0$, $d(j) := +\infty$ per $j \neq s$; tutti i nodi sono temporanei.
2 **Passo 1 (selezione).** Se nessun nodo temporaneo ha un'etichetta finita, `STOP`.
3 Altrimenti rendere permanente il nodo temporaneo i con $d(i)$ minima (quello di indice minimo, se ce n'è più di uno).
4 **Passo 2 (aggiornamento).** Per ogni arco $(i, j) \in \delta^+(i)$ con j temporaneo: se $d(i) + d_{ij} < d(j)$, porre $d(j) := d(i) + d_{ij}$ e $pred(j) := i$.
5 Andare al Passo 1.

Teorema 3.22: correttezza dell'algoritmo di Dijkstra

Se $d_{ij} \geq 0$ per ogni arco, a ogni iterazione dell'algoritmo di Dijkstra:

(a) ogni nodo permanente i ha $d(i) = \tilde{d}(i)$;

(b) ogni nodo temporaneo j ha come $d(j)$ la lunghezza minima di un cammino da s a j i cui nodi diversi da j sono tutti permanenti ($+\infty$ se non ce n'è nessuno).

All'arresto, $d(j) = \tilde{d}(j)$ per ogni $j \in N$.

Dimostrazione. Entrambe le proprietà valgono dopo il Passo 0, quando non ci sono nodi permanenti. Sia i il nodo scelto al Passo 1.

1. $d(i) = \tilde{d}(i)$. Sia P un cammino qualsiasi da s a i, e sia h il suo primo nodo temporaneo. Il tratto di P fino a h ha lunghezza $\geq d(h)$ per (b), e il resto ha lunghezza ≥ 0, perché $d \geq 0$. Quindi $d(P) \geq d(h) \geq d(i)$, e per (b) il valore $d(i)$ è la lunghezza di un cammino: $d(i) = \tilde{d}(i)$.
2. *(b) dopo il Passo 2.* Sia j temporaneo e Q un cammino verso j i cui altri nodi sono permanenti, e sia h il nodo che precede j. Se $h = i$, allora $d(Q) \geq \tilde{d}(i) + d_{ij} = d(i) + d_{ij}$, il valore controllato al Passo 2.
3. Se $h \neq i$, allora h è diventato permanente prima di i, attraverso un cammino di nodi permanenti di lunghezza $\tilde{d}(h)$; quando h è stato scelto, il Passo 2 ha dato $d(j) \leq \tilde{d}(h) + d_{hj} \leq d(Q)$.

Quindi il nuovo minimo è $\min\{d(j),\, d(i)+d_{ij}\}$, realizzato da un cammino, ed è l'etichetta lasciata dal Passo 2. All'arresto, i nodi temporanei hanno $d(j)=+\infty$: per (b) nessun arco va dall'insieme dei nodi permanenti verso di loro, quindi non sono raggiungibili e $\tilde{d}(j)=+\infty$. □

- Per il problema s–t l'algoritmo può fermarsi appena t diventa permanente, per (a).
- Gli archi $(pred(j), j)$ formano un albero dei cammini minimi: $pred(j)$ diventa permanente prima di j, e quando viene posto per l'ultima volta vale $d(j)=d(pred(j))+d_{pred(j)\,j}$, con entrambe le etichette definitive.

Teorema 3.23: tempo di esecuzione dell'algoritmo di Dijkstra

Con le etichette in un vettore, l'algoritmo di Dijkstra richiede tempo $O(n^2)$.

Dimostrazione. Le iterazioni sono al più n, e il Passo 1 esamina le etichette temporanee in tempo $O(n)$. Il Passo 2 esamina la stella uscente del nodo scelto, e ogni nodo viene scelto una volta, quindi tutti gli aggiornamenti richiedono tempo $O(m)$. Il totale è $O(n^2+m)=O(n^2)$, perché $m \le n(n-1)$. □

- Con le etichette temporanee in un heap binario, il Passo 1 e ogni diminuzione di un'etichetta richiedono tempo $O(\log n)$: il totale è $O((n+m)\log n)$, migliore di $O(n^2)$ sui grafi sparsi.

3.6.1 L'algoritmo di Dijkstra sul grafo di riferimento

Esempio 3.9: Dijkstra dal nodo 1

Eseguiamo l'algoritmo con le lunghezze dell'Esempio 3.1. Ogni riga è un'iterazione: il nodo scelto diventa permanente (etichetta in grassetto), e accanto a un'etichetta che è diminuita c'è, tra parentesi, il nuovo predecessore.

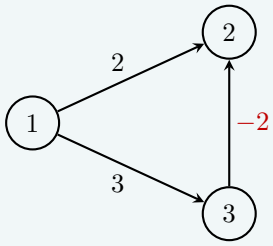
iterazione	nodo scelto i	$d(1)$	$d(2)$	$d(3)$	$d(4)$	$d(5)$	$d(6)$
1	1	**0**	7 (1)	2 (1)	∞	∞	∞
2	3	**0**	5 (3)	**2**	∞	10 (3)	∞
3	2	**0**	**5**	**2**	8 (2)	10	∞
4	4	**0**	**5**	**2**	**8**	10	13 (4)
5	5	**0**	**5**	**2**	**8**	**10**	13
6	6	**0**	**5**	**2**	**8**	**10**	**13**

- Nell'iterazione 2 l'etichetta del nodo 2 scende da 7 a $2+3=5$; nell'iterazione 5 l'arco $(5,6)$ offre $10+7=17>13$, e l'arco $(5,4)$ termina in un nodo permanente.
- I nodi diventano permanenti nell'ordine $1,3,2,4,5,6$, con $\tilde{d}=(0,5,2,8,10,13)$; i predecessori danno l'albero dei cammini minimi dell'Esempio 3.3.

3.6.2 Lunghezze negative

Esempio 3.10: un arco negativo fa fallire l'algoritmo di Dijkstra

Nel grafo qui sotto, con $s = 1$, l'algoritmo rende permanente il nodo 2 con $d(2) = 2$, poi il nodo 3 con $d(3) = 3$; l'arco $(3, 2)$ termina in un nodo permanente e viene ignorato. Il cammino $1, 3, 2$ ha lunghezza $3 - 2 = 1 < 2$.



Sul grafo di riferimento con le lunghezze d' dell'Esempio 3.7 ($d'_{54} = -3$), l'algoritmo di Dijkstra rende permanenti i nodi nell'ordine $1, 3, 2, 4, 5, 6$ e restituisce $d(4) = 8$ e $d(6) = 13$: il nodo 4 diventa permanente prima del nodo 5, quindi il cammino più corto $1, 3, 5, 4$, di lunghezza 7, sfugge all'algoritmo, e con esso il cammino verso 6 di lunghezza 12.

- La dimostrazione del Teorema 3.22 usa $d \geq 0$ in un solo punto: il resto di P dopo il suo primo nodo temporaneo ha lunghezza ≥ 0. Un arco negativo invalida proprio questo punto.

Con lunghezze non negative, l'algoritmo di Dijkstra rende permanente, a ogni iterazione, il nodo temporaneo con l'etichetta minima, che da quel momento è definitiva; richiede tempo $O(n^2)$. Con un arco negativo può sbagliare, e si usa invece l'algoritmo di Bellman–Ford.

3.7 Errori tipici

Attenzione

- **Eseguire l'algoritmo di Dijkstra con un arco negativo.** Le sue etichette possono essere sbagliate, come con $d'_{54} = -3$ sul grafo di riferimento; usare Bellman–Ford, oppure l'ordine topologico se il grafo è aciclico.
- **Sommare una costante a ogni lunghezza.** Rende le lunghezze non negative ma cambia i cammini minimi: un cammino con k archi cresce di k volte la costante.
- **Invertire il segno dei potenziali.** Con $+1$ nella coda di $\boldsymbol{B}$ i vincoli duali sono $\pi_i - \pi_j \leq d_{ij}$ e i potenziali ottimi sono $\pi_i = -\tilde{d}(i)$.
- **Saltare il test della passata n.** Dopo $n - 1$ passate le etichette sono le distanze solo in assenza di cicli negativi; un cambiamento nella passata n ne rivela uno.
- **Aspettarsi che Bellman–Ford veda ogni ciclo negativo.** Rileva solo i cicli raggiungibili da s, mentre un ciclo negativo in un punto qualsiasi di G rende illimitato il modello LP.

- **Aggiornare un nodo permanente nell'algoritmo di Dijkstra.** La sua etichetta è definitiva; si aggiornano solo i nodi temporanei.
- **Cercare un ordine topologico in un grafo con cicli.** Esiste solo nei grafi aciclici: il grafo di riferimento non ne ha.

3.8 In sintesi

In sintesi

- La distanza $\tilde{d}(j)$ è la lunghezza di un cammino minimo da s a j; è ben definita quando nessun ciclo negativo è raggiungibile da s (A2).
- I sottocammini dei cammini minimi sono minimi; le distanze soddisfano le equazioni di Bellman, ed esiste un albero dei cammini minimi.
- Potenziali ammissibili, $\bar{c}_{ij} = d_{ij} - \pi_i + \pi_j \geq 0$, e un cammino di archi stretti certificano l'ottimalità.
- Il modello LP è un flusso unitario, $\boldsymbol{B}\boldsymbol{x} = \boldsymbol{b}$, $\boldsymbol{x} \geq \boldsymbol{0}$; il suo duale è $\max\{\pi_s - \pi_t : \pi_i - \pi_j \leq d_{ij}\}$, con ottimo in $\pi_i = -\tilde{d}(i)$.
- Grafi aciclici: ordine topologico, $O(n + m)$, lunghezze qualsiasi. Bellman–Ford: $O(n\,m)$, lunghezze qualsiasi, rileva i cicli negativi. Dijkstra: $O(n^2)$, lunghezze non negative.

Il grafo di riferimento in breve.

lunghezze d (Dijkstra)	$\tilde{d} = (0, 5, 2, 8, 10, 13)$, ordine $1, 3, 2, 4, 5, 6$
modelli LP	$z(\mathrm{SP}(1, 6)) = 13$, $z(\mathrm{SP}(1)) = 38$
lunghezze d', $d'_{54} = -3$ (Bellman–Ford)	$\tilde{d} = (0, 5, 2, 7, 10, 12)$, 5 passate
lunghezze d'', $d''_{54} = -10$	ciclo negativo $3, 5, 4, 3$ di lunghezza -1
grafo aciclico, arco $(3, 4)$	ordine $1, 3, 2, 5, 4, 6$; $\tilde{d} = (0, 5, 2, 3, 10, 8)$